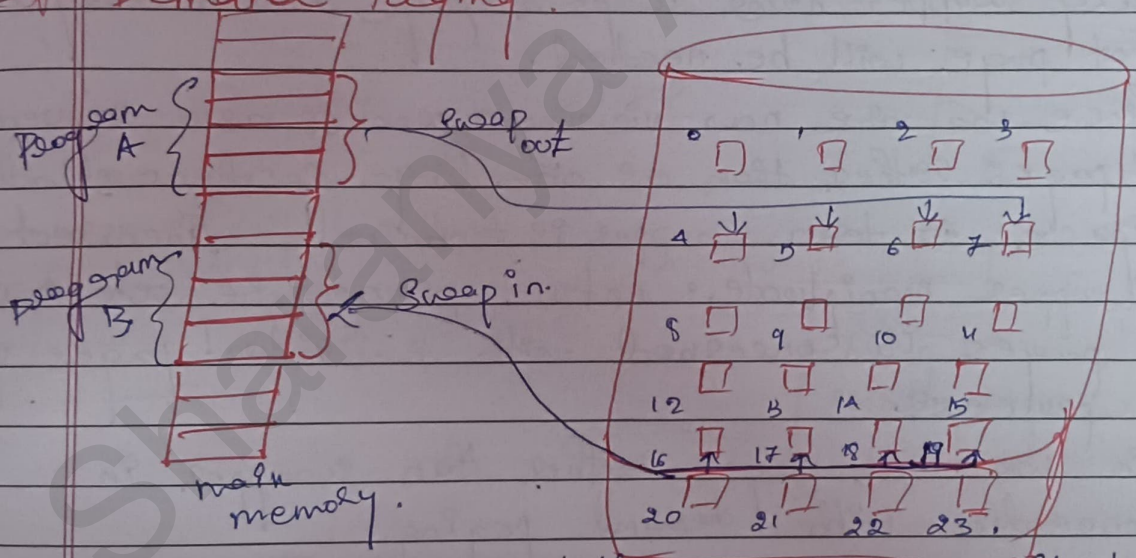
# OS – UNIT – 3

## 1. VIRTUAL MEMORY

- Virtual memory is a technique that allows the execution of processes that are not completely in memory.
- Advantage of this is that programs can be larger than physical memory.
- Virtual memory involves the separation of logical memory as perceived by users from Physical Memory.
- This separation allows an extremely large virtual memory to be provided for programs when only a smaller physical memory is available.

### Demand Paging:




- Consider an executable program might be loaded from disk into memory.
- One option is to load entire program in physical memory at program execution time.
- But problem is that we may initially not need the entire program in memory.
- Loading the entire program into memory results in loading the executable code for all options, regardless of whether an option is ultimately selected by the user or not.

→ An alternative strategy is to initially load pages only as they are needed, this technique is known as Demand paging & commonly used in virtual memory systems.
→ With this pages are loaded only when they are demanded during program execution, pages that are never accessed are thus never loaded into physical memory.
→ This is similar to a paging with swapping where process resides in secondary memory.
→ When we want to execute the process we swap it into memory rather than swapping the entire process into memory. However we use a LAZY SWAPPER.
→ Lazy swapper never swaps a page into memory unless that page will be needed.
→ Since we are now viewing a process as a sequence of pages rather than as one large contiguous address space so term swapper is technically incorrect.
→ Swapper manipulate's entire procedure whereas a pager is concerned with individual pages of a process.
→ So we use pages rather than swapper in connection with demand paging.

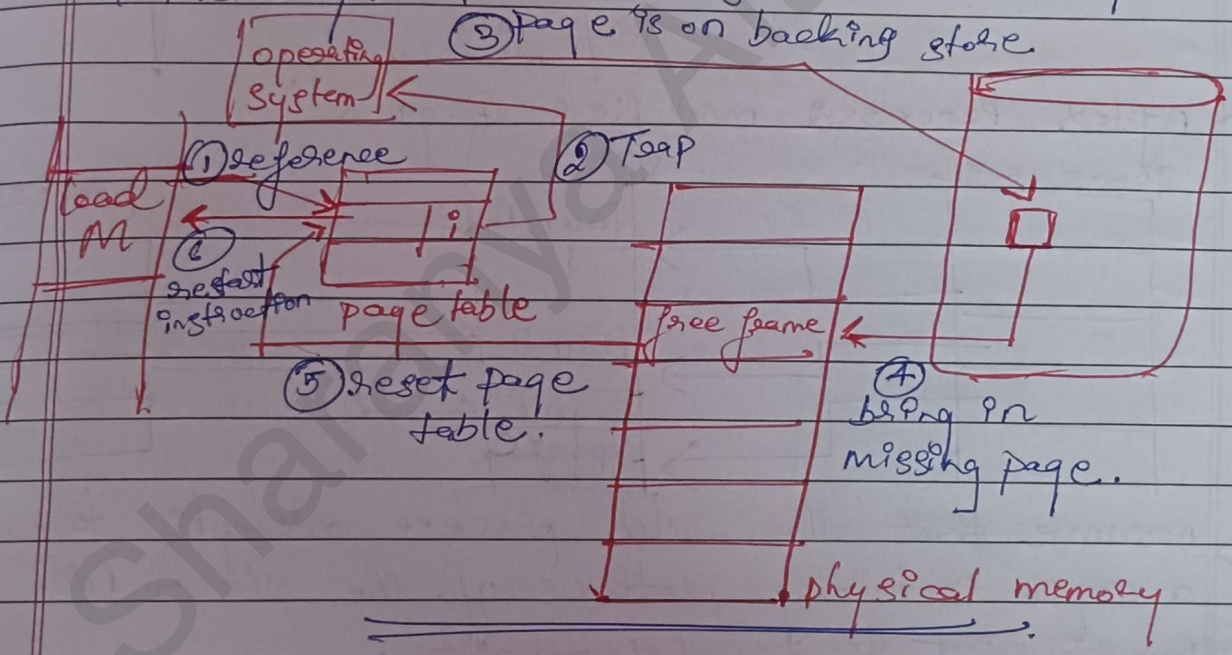
## * Page Fault:

→ It is a trap where process tries to access a page that was not brought into memory
→ This trap is the result of the OS's failure to bring the desired page into memory.

## * Steps / Procedure for handling the Page Fault:

① We check an internal table for the process to determine whether the reference was a valid or invalid memory access.

(2) If the reference was invalid, we terminate the process. If it is valid & not yet brought in that page we now page it.
(3) We find a free frame by taking one from the free frame list.
(4) We schedule a disk operation to read the desired page into the newly allocated frame.
(5) When the disk read is complete we modify the internal table kept with the process & the page table to indicate that page is now in memory.
(6) We restart the instruction that was interrupted by the trap. The process can now access the page as though it had always been in memory.



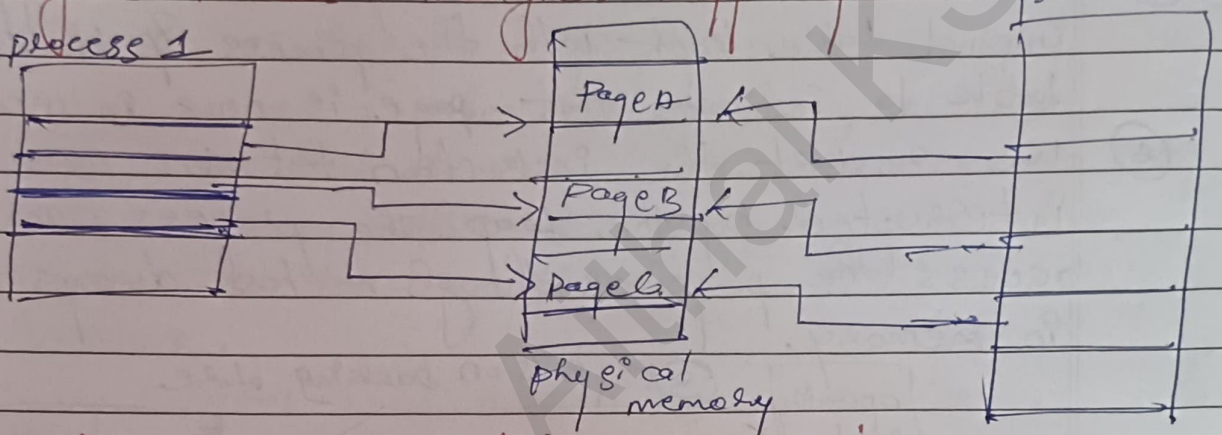

* Pure Demand Paging: Never bring a page into memory until it is required.
→ Page Table: This table has the ability to mark an entry valid/invalid bit or special value of protected bits.
→ Secondary memory: This memory hold those pages that are not present in main memory.

## Copy on Write:

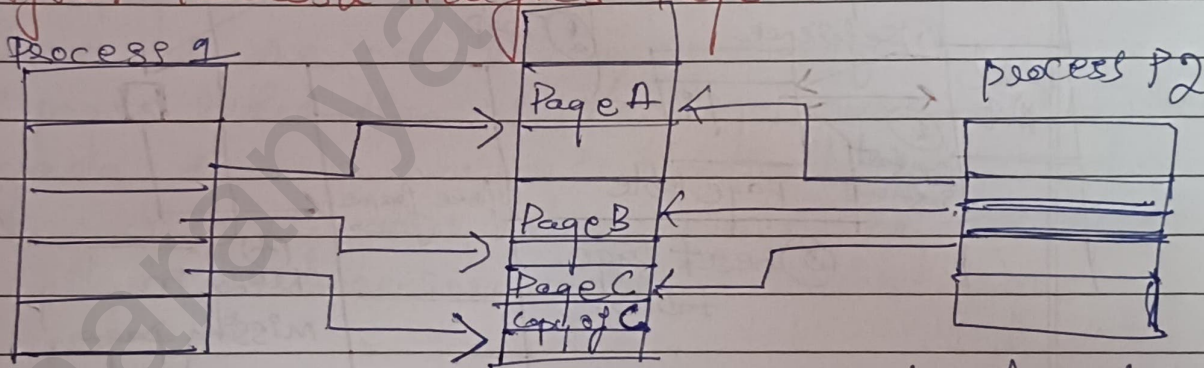
→ This allows the parent & child process initially to share the same pages in memory.

→ These pages are marked as copy on write pages meaning that if either process writes to a shared page a copy of the shared page is created.

→ Before Process 1 modifies Page C:




→ After Process 1 modifies Page C:




→ Assume that the child process attempts to modify a page containing portions of the stack with the pages set to be copy-write.

→ The OS will then create a copy of this pages mapping it to the address space of the child process.

→ The child process will then modify its copied page & not the page belonging to the parent process.

→ When copy-on write technique is used only the pages that are modified by either process are copied all unmodified pages can be shared by the parent & child processes.

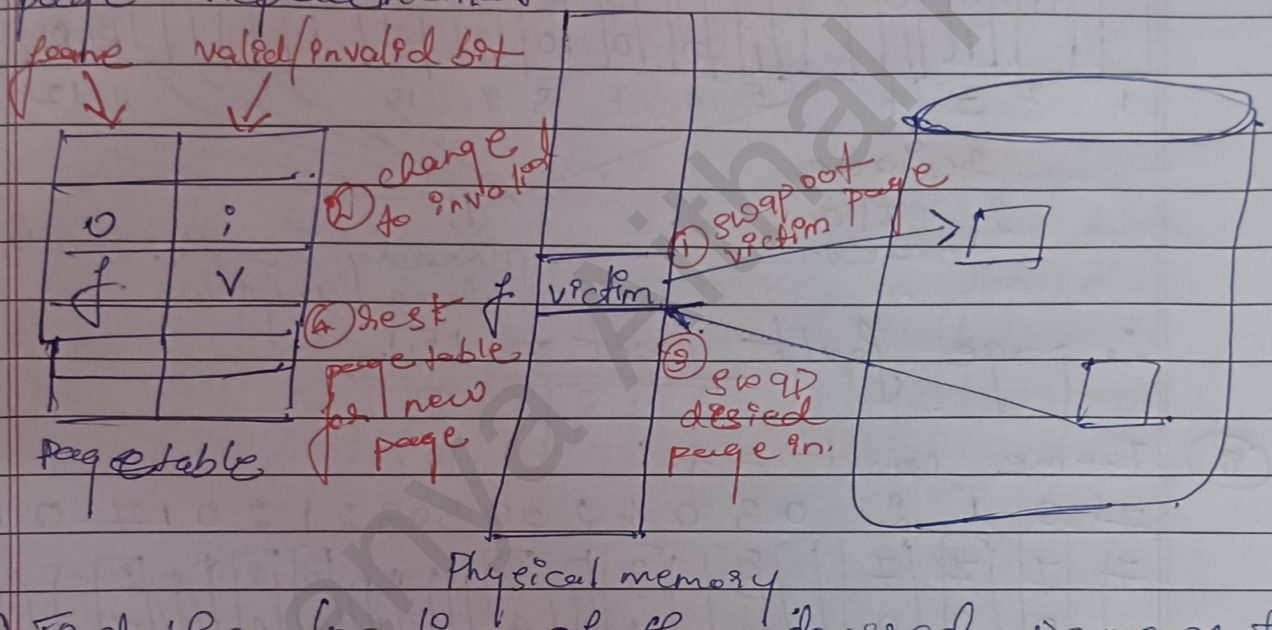
→ Only pages that can be modified need to be marked as copy-on-write.

## Page Replacement:

→ If no frame is free, we find one that is not currently being used & free it.

→ We can free a frame by writing its contents to swap space & changing the page table.

→ We can now use the freed frame to hold the page for which the process faulted. We modify the page fault-service routine to include page replacement.




(1) Find the location of the desired page on the disk

(2) Find a free frame
- (i) if there is a free frame use it.
- (ii) if there is no free frame use a page replacement algorithm to select a victim page.
- (iii) write the victim frame.

(3) Read the desired page into the newly freed frame, change the page & frame tables.

(4) Restart the user process.

→ Frame allocation Algorithm: determines
- (i) how many frames to give each process.
- (ii) which frames to replace.

→ Page Replacement Algorithm:
wants lowest page fault rate on both access & re-access.

## Page Replacement Algorithms:

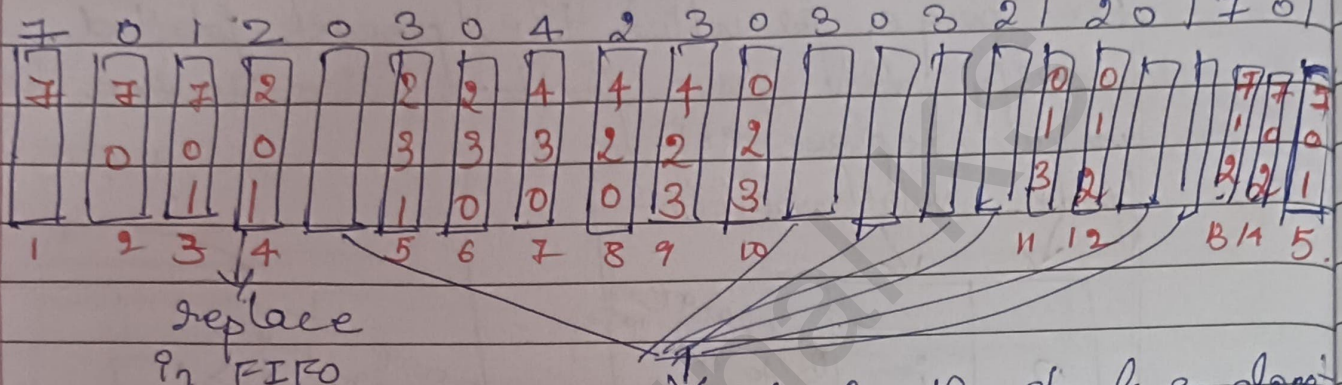
Consider the following reference string:

7 0 1 2 0 3 0 4 2 3 0 3 0 3 2 1 2 0 1 7 0 1

Find page fault count.

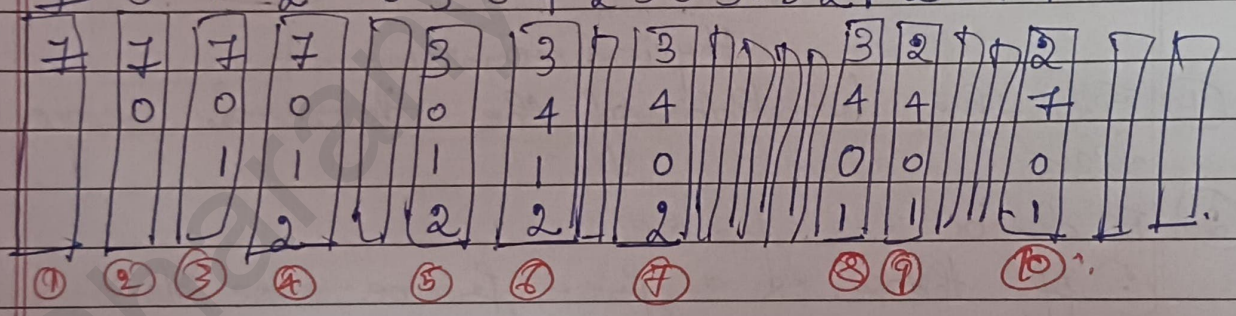
(1) FIFO (First In First out) Algorithm:

(i) Frame count = 3.




∴ page fault count = 15.

(ii) Frame count = 4:



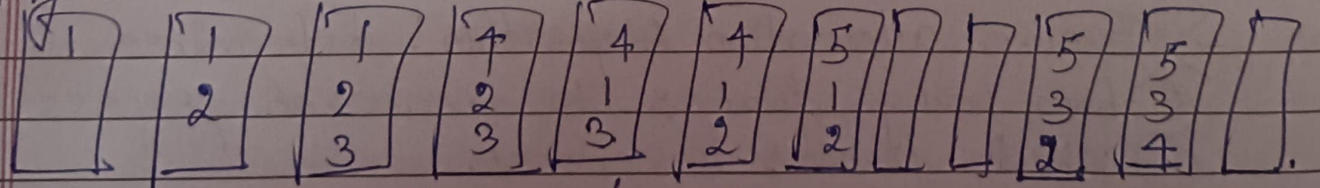

∴ page fault count = 10.

(iii) Balad's Anomaly: Adding more frames can cause more page faults.

~~Some book for FCFS algorithm.~~

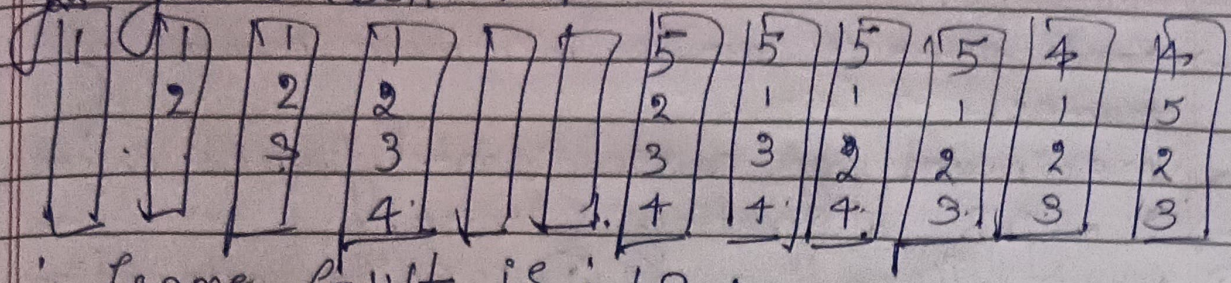
⇒ Consider: 1 2 3 4 1 2 5 1 2 3 4 5.

for frame count = 3.



∴ Page fault count = 9.

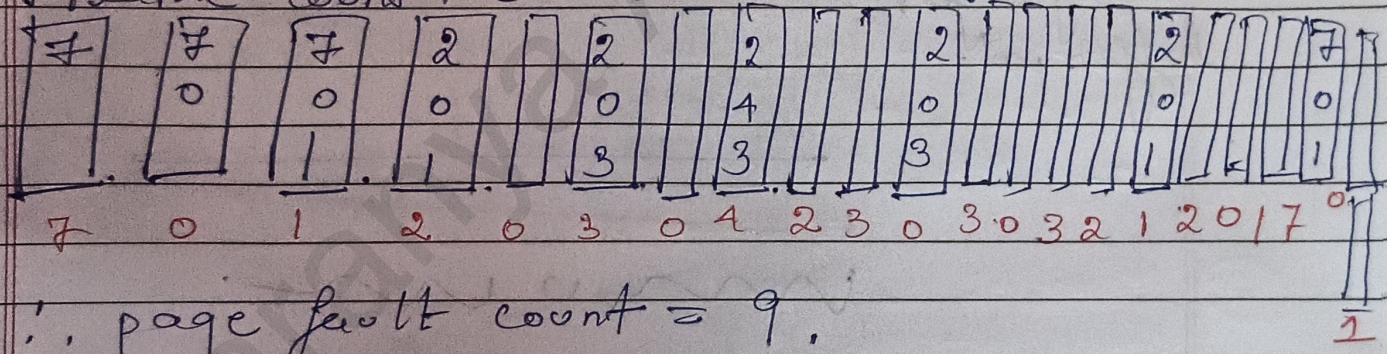
for frame count : 4



∴ frame fault is : 10.

We observe that as frame count is increased the frame fault also increase. Hence it is Belady's Anomaly.

(2) Optimal Page Replacement Algorithm:
Replace the page that will not be used for longest period of time in future.
Frame count : 3.

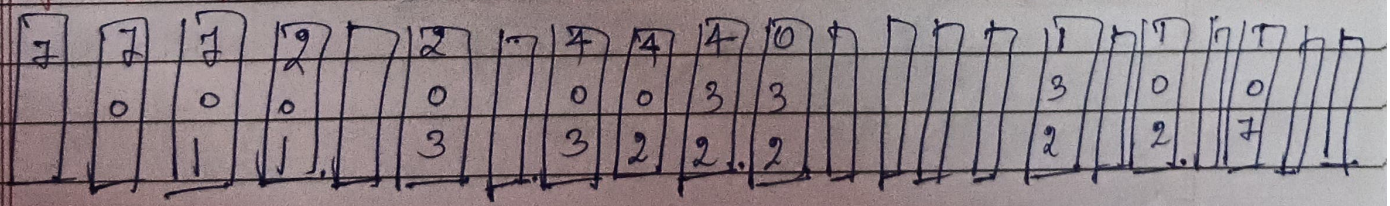


7 0 1 2 0 3 0 4 2 3 0 3 0 3 2 1 2 0 1 7 0 1

∴ page fault count = 9.

(3) LRU (Least Recently used) Page Replacement:
→ Replace the page that is not used in the most amount of time. (Frame count : 3)

7 0 1 2 0 3 0 4 2 3 0 3 0 3 2 1 2 0 1 7 0 1



∴ page fault count = 12.

## Allocation of Frames:

→ Each process needs minimum number of frames.

→ Maximum of course is total frames in the system.

→ 2 major allocation schemes:

① fixed allocation & ② Priority allocation.

① fixed allocation
- Equal allocation
  - 100 frames & 5 process
  - give each process 20 frame. & keep few for buffer pool.
- Proportional allocation
  - allocate acc to size of process.

② Priority allocation
- use proportional allocation but use priority rather than size.

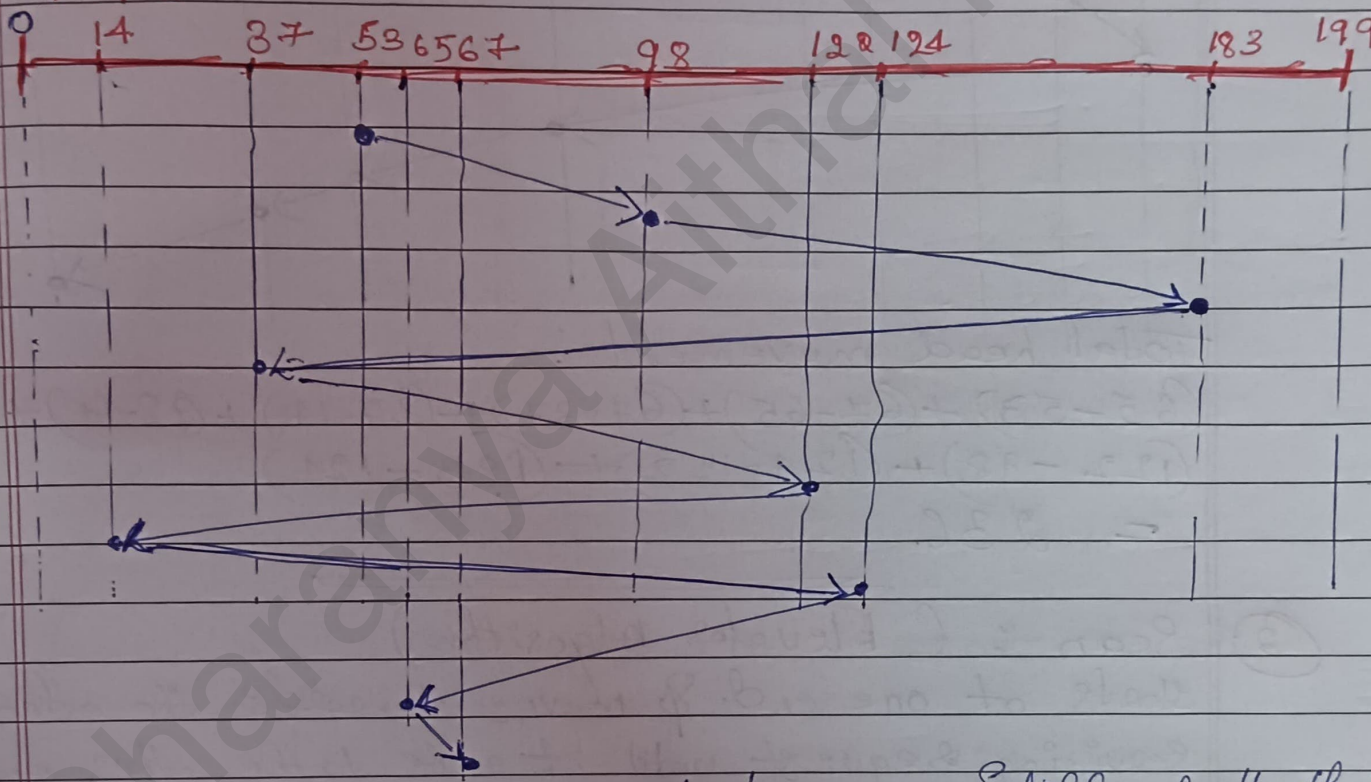
# ② DISK SCHEDULING

→ We illustrate scheduling algorithm with a request queue (0-199)

queue: 98, 183, 37, 122, 14, 124, 65, 67.

Header pointer: 53.

① FCFS:




Total movement of disk = sum {diff of all the values of each point}

Total movement of disk = (53-98) + (98-183) + (183-37) + (122-37) + (122-14) + (124-14) + (124-53) + (65-53) + (67-65)

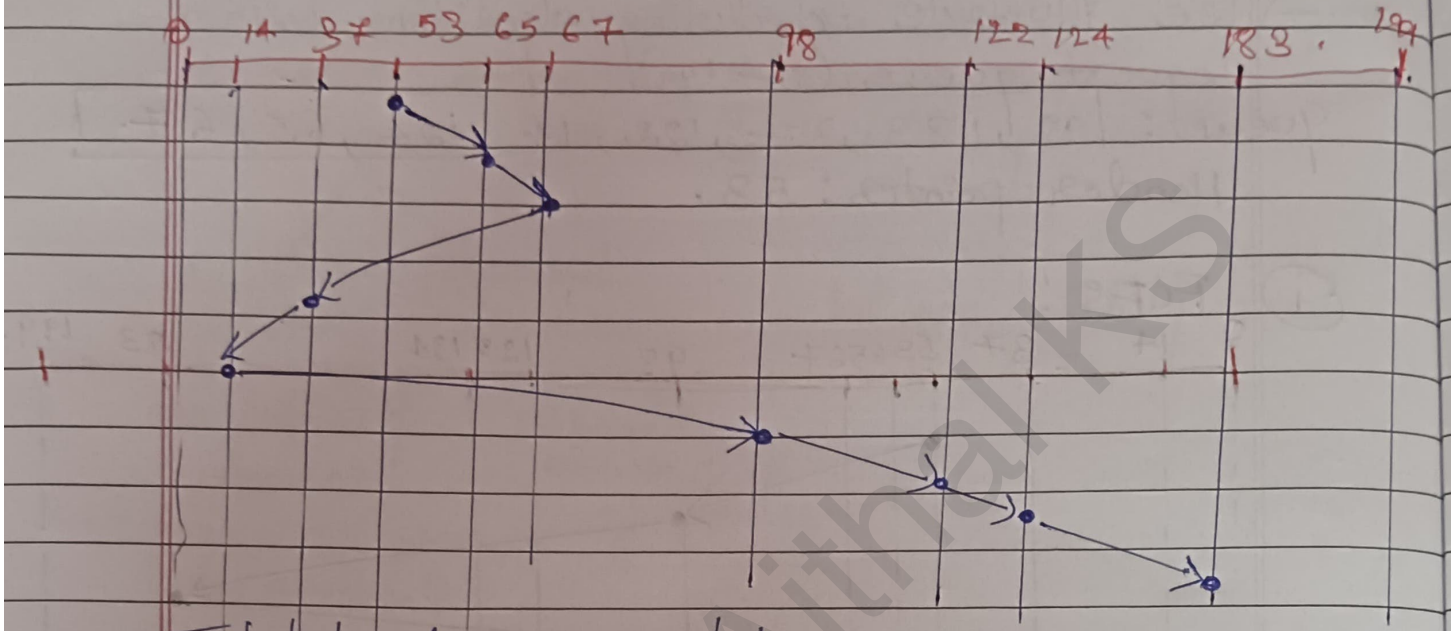
= 45 + 85 + 146 + 85 + 108 + 110 + 59 + 12 + 02.

= 640.

② SSTF {Shortest Seek Time First}

queue : 98, 183, 37, 122, 14, 124, 65, 67

head start : 53.




Total head movement :

(65-53) + (67-65) + (67-37) + (37-14) + (98-14) + (122-98) + (124-122) + (183-124)

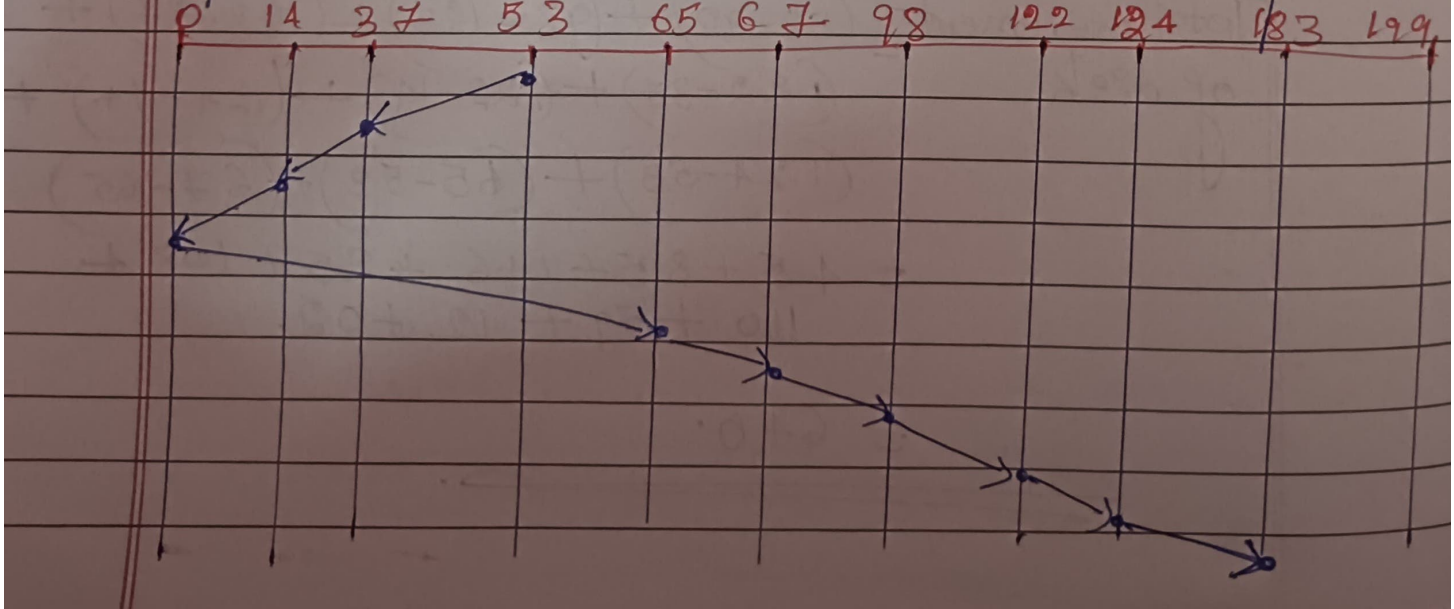
= 236.

---

③ Scan : (Elevator Algorithm)

Starts at one end & moves towards the other end servicing request until it gets to the other end of disk, where the head movements is reversed & servicing continues.

→ Go to that side which has least request.



Total head movement:

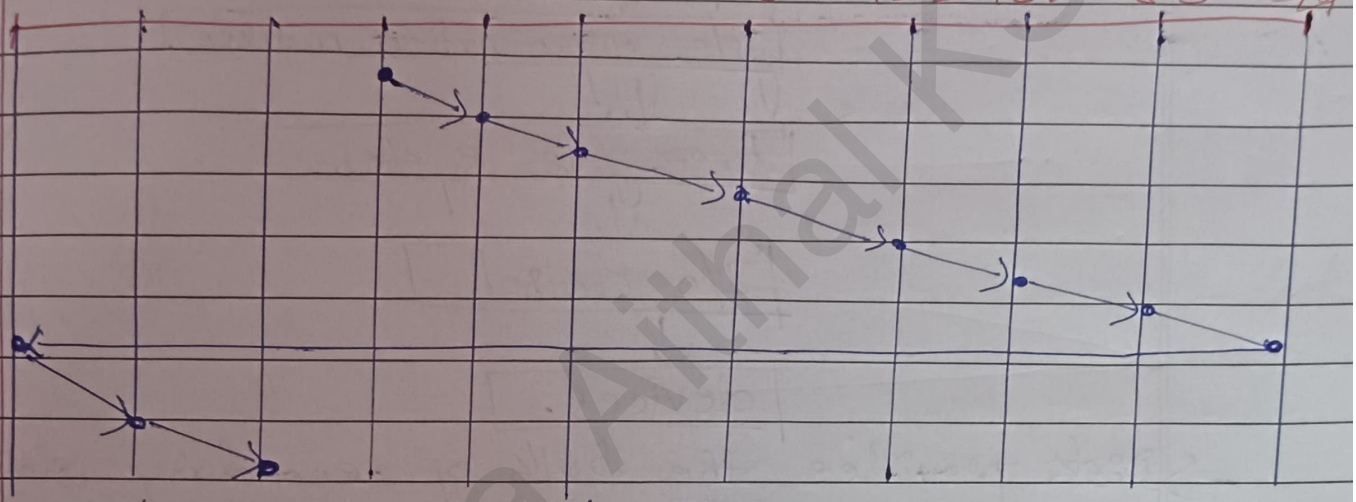
$(53-37)+(37-14)+(14-0)+(65-0)+(67-65)+(98-67)+(122-98)+(124-122)+(183-124)$

$= 236.$

⇒ Go to ~~max~~ request side.

(4) CScan: once you reach end go to other end & then restart.

0 14 37 53 65 67 98 122 124 183 199

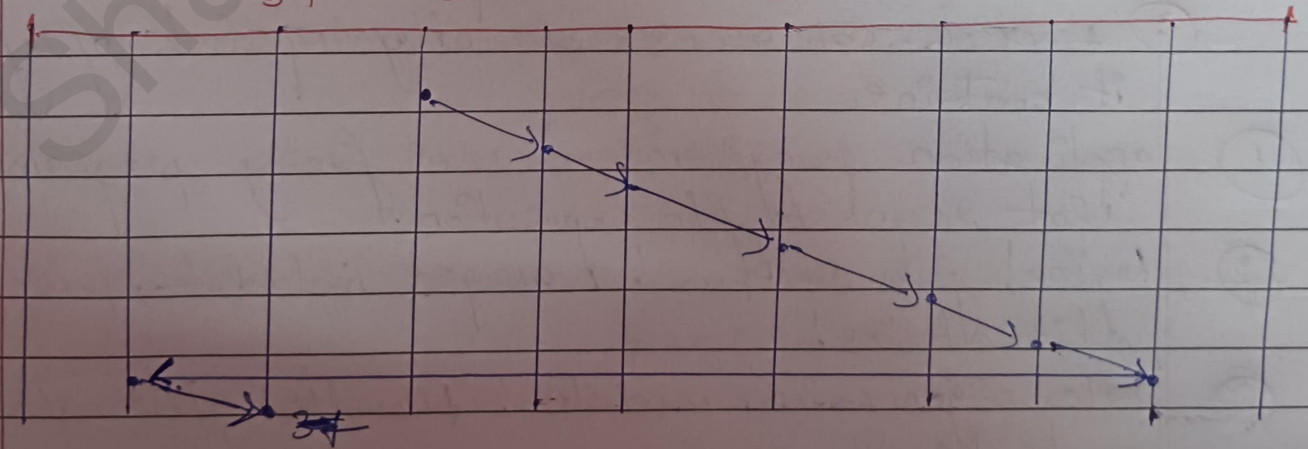


Total head movement: 382.

(5) CLook:

Same as CScan but we do not touch any end points (0/199).

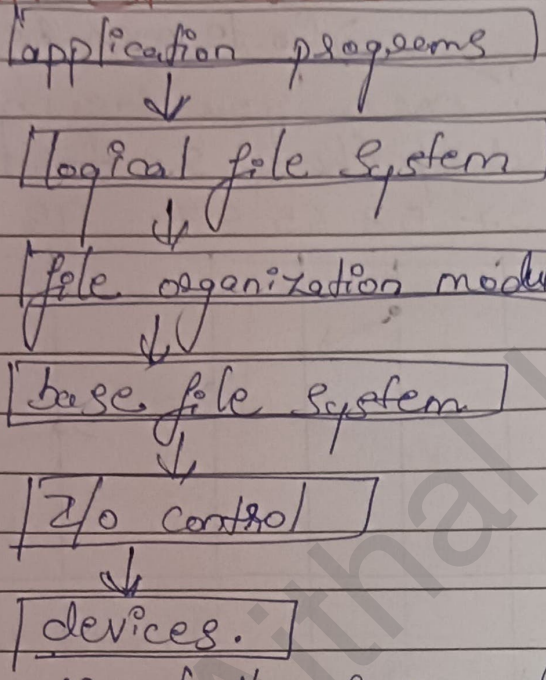
0 14 37 53 65 67 98 122 124 183 199



Total head movement = 322.

# ③ FILE SYSTEM

* File System Structure:




- Disk provides the bulk of secondary storage on which a file system is maintained.
- They have 2 characteristics that makes them a convienient medium for storing multiple files.
  1. A disk can be rewritten in place. It is possible to read from the disk, modify the block & write it back into same place.
  2. A disk can access directly any block of information it contains

1. application programs: user facing programs that request file operation.
2. Logical file system: manages metadata, protection & directories.
3. File organization module: Handles file storage methods & mapping
4. Base file system: Deals with physical block operations
5. I/O control: uses drivers & interrupt handlers to communicate with hardware.
6. Devices: Actual physical storages.

## File Control Block: (FCB)

| file permissions |
| --- |
| file dates (create, access, write) |
| file owner, group, ACL |
| file size |
| file data, blocks/pointers to file data blocks |

→ OS maintains FCB per file which contains many details about the file.

## Allocation Methods:

→ Refers to how disk blocks are allocated to files.

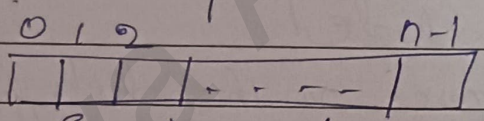
① **Contiguous**: Each file consists of contiguous block
→ Best performance in most cases.
→ Simple only starting location & length required.
→ Problems include
(i) finding space on the disk for a file
(ii) knowing size of file.
(iii) External fragmentation.

② **Linked**: Each file is a linked list of blocks.
→ file ends at nil pointer.
→ No external fragmentation
→ Each block contains pointer to next block.
→ Free space management system called when new block needed.
→ Improve efficiency by clustering blocks into groups but increases internal fragmentation.
→ Reliability can be a problem
→ Locating a block can need many I/O & disk seeks.

③ File Allocation Table (FAT) Allocation:
→ Beginning of volume has table, indexed by block numbers
→ Much like a linked list but faster on disk & cacheable.
→ new blocks allocation simple.

④ Indexed Allocation:
→ Each file has its own index block of pointers to its data blocks.

## Free Space Management:
→ File system maintains free space list to track available blocks/clusters.
→ Bit vector / Bit map:



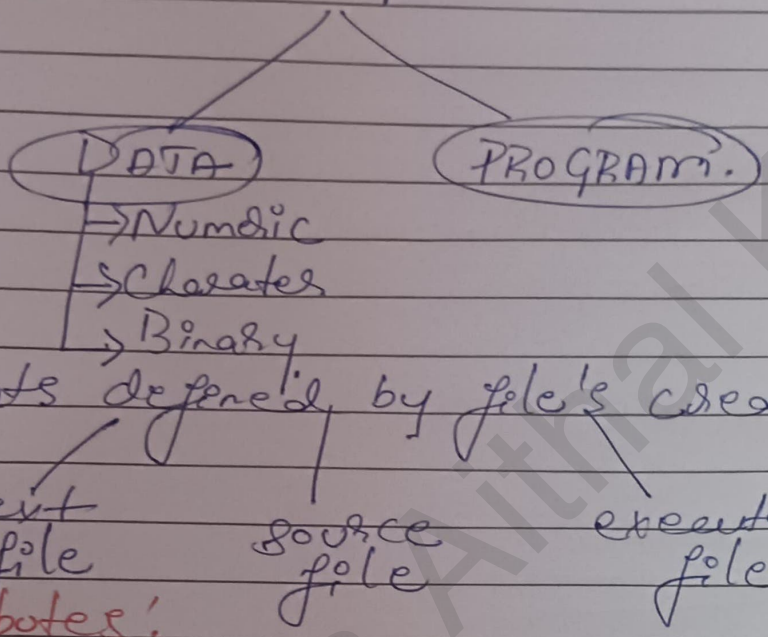

$$bit[i] = \begin{cases} 1 \Rightarrow block[i] \text{ free} \\ 0 \Rightarrow block[i] \text{ occupied.} \end{cases}$$

→ Bit map requires extra space.
→ Easy to get contiguous files.
→ Grouping: modified linked list to store address of next n-1 free blocks in first free block + a pointer to next block that contains free-block pointers.
→ Counting: Because space is frequently contiguously used & freed, with contiguous allocation, extents or clustering
→ keep address of 1st free block & count of following free blocks
→ Free space list then entries containing addresses & counts.

# * File, types, attributes & operation of file:

→ File is a named collection of data stored on secondary storage.

⇒ Types:




→ Contents defined by file's creator.

⇒ Attributes:

1. Name: only information kept in human-readable form
2. Identifier: unique tag identifies file within file sys.
3. Type: needed for systems that supports diff types.
4. Location: pointer to file location on device.
5. Size: current file size.
6. Protection: controls who can read, write, execute.
7. Time, date & user identification: data for protection, security & usage monitoring.

⇒ File Operations:

1. Create
2. write — at write pointer location
3. Read — at read pointer location
4. Reposition within file — seek.
5. Delete
6. Truncate.
7. Open($F_i$): search directory structure on disk for entry $F_i$ & move content to entry to memory.
8. close($F_i$): move content of $F_i$ in memory to directory on disk.